Notification Priority Classification Using Semantic Analysis

Oliver Jackson Holmes
Computer Science and Artificial Intelligence
IE University
Madrid, Spain
oholmes.ieu2023@student.ie.edu

Denis Sokolov Ivanov
Computer Science and Artificial Intelligence
IE University
Madrid, Spain
denis_sokolov@student.ie.edu

Sofia Claudia Bonoan San José
Computer Science and Artificial Intelligence
IE University
Madrid, Spain
sbonoan.ieu2023@student.ie.edu

Gonzalo Nicolás Fernández de Córdoba Javronich
Computer Science and Artificial Intelligence
IE University
Madrid, Spain
gfernandezde.ieu2023@student.ie.edu

***Abstract*—Smartphones generate many notifications, but not all are equally important. This project builds a text-based classifier that assigns each notification a priority level (High, Medium, Low) based on what the message says. We curated a dataset of 737 real notifications and applied a privacy-preserving anonymization pipeline. We create stratified train/validation/test splits and fine-tune transformer models (BERT/RoBERTa). We evaluate performance using macro-F1 and a top-k ranking check to verify that urgent notifications appear first.**

***Index Terms*—notification management, priority classification, NLP, transformers**

I. Introduction

Smartphones and laptops send people dozens of notifications every day. While some are important (for example a security warning or a time-sensitive reminder), many are not urgent. When everything is shown in the same way, users can miss important alerts or get distracted by low-value ones. Past research shows that interruptions can reduce performance, increase errors, and affect mood [1], [8]. Notifications can also increase inattention and make it harder to focus [12]. Because of this, deciding which notifications deserve attention first is an important problem.

A large body of work has studied how people interact with notifications and when they are receptive to them. Prior studies measured notification behavior in real settings and at large scale [3], [4], and examined user receptivity and response patterns [5]. Other work looked at content importance and how users decide what matters [6]. There are also approaches that learn user preferences to support smarter notification management [9]. However, many systems focus on *when* to interrupt (interruptibility and receptivity) rather than directly predicting the *urgency* of the notification from its text. Recently, urgency has been studied more directly in mobile push notifications [7], but building a practical content-based prioritization system that can rank notifications remains a useful and open direction.

In this project, we focus on **content-based urgency detection**. Our research question is: *Can we predict the urgency level of a notification using only its text content (title + body), and use that prediction to rank notifications so urgent ones appear first?* We use three urgency levels (High, Medium, Low), inspired by prior work on urgency and notification management [6], [7]. We also include a ranking step because the real goal is not only to assign a label, but to surface the most urgent items first.

Our hypothesis is that transformer-based language models will classify urgency more accurately than simpler text baselines because they capture context in short messages (for example phrases like "verify now", "starting in 10 minutes", or "payment failed") [13], [14]. Our hypothesis is that transformer-based language models will classify urgency more accurately than simpler text baselines because they capture context in short messages (for example phrases like "verify now", "starting in 10 minutes", or "payment failed") [13], [14]. To test this, we curated a labeled dataset of 737 real notifications with text content and context fields (e.g., app name and notification type). We apply privacy-preserving anonymization, create stratified train/validation/test splits, and fine-tune transformer models (BERT/RoBERTa). We evaluate with macro-F1 (important for class imbalance) and a top-k ranking check to measure whether truly urgent notifications appear near the top. Implementation is done using standard tooling for transformer training and evaluation [15].

This work connects two important ideas from the literature: notifications can harm focus when they are poorly managed [1], [12], and notification importance varies by content and context [6], [7]. By building and evaluating a text-based urgency classifier and ranking method, we aim to show a practical way to reduce unnecessary distraction while still surfacing high-priority information.

The remainder of the paper is organized as follows. Section II reviews related work on notifications, attention, and

urgency. Section III describes our dataset, labeling process, models, and evaluation methods. Section IV will report experimental results, and Section V will discuss limitations, bias considerations, and future work.

II. Related Literature

Notifications are useful, but they can also interrupt users and reduce performance. Controlled studies show that interruptions increase completion time and errors and can worsen user experience [1]. Field work on alerts shows users value awareness, but frequent notifications still disrupt focus [2]. On mobile, in-situ and large-scale studies confirm high notification volume and frequent dismissals, suggesting many alerts are low value or poorly timed [3], [4]. Experimental results also link notifications to reduced attention outcomes, motivating better filtering [12].

Most prior work improves notification handling by predicting *interruptibility* or *receptivity* from context and behavior. For example, delivering messages at activity transitions can reduce perceived interruption [8]. Receptivity models show that context and individual differences affect response and disruption [5]. Systems like PrefMiner learn user-specific rules for notification management using interaction history [9]. Other studies predict attentiveness to mobile messages [10] or identify moments when users are open to engage with suggested content [11]. These approaches help with timing, but they often use notification text in a limited way.

For *priority* (high/medium/low), the message content is critical. Work on content importance shows that interaction behavior does not always match what users consider important, and combining context with message content improves filtering [6]. Related work on urgency in push notifications also shows urgency levels can be defined and predicted using notification text [7]. Based on this, our project focuses on semantic priority classification using transformer-based text models. We use pretrained encoders such as BERT and RoBERTa because they capture meaning beyond keywords and perform well for short-text classification [13], [14]. We implement training and evaluation using the Transformers library [15]. In our labeling, *high* means urgent/time-sensitive, *medium* means important but not urgent, and *low* means informational.

III. Experimental Methodology / Methods (Algorithms and Implementation)

This section explains how we built the dataset and how we will train and evaluate a system that predicts notification urgency from text. Our goal is to classify each notification into an urgency level (High/Medium/Low) and then rank notifications so urgent ones appear first.

At the start of the project, we planned to use a public dataset we found on Kaggle. After inspecting it, we realized it was not suitable for our goal because it did not include the full notification text needed for semantic analysis (it mainly contained metadata fields). Since our project depends on understanding the meaning of the notification message, we decided to curate our own dataset with real notification text and context fields. This change made the experiments match our research question and allowed us to build a realistic urgency classifier.

TABLE I
Dataset summary after cleaning and anonymization.

Total notifications	737
High / Medium / Low	258 / 196 / 283
Total words	6,887
Unique words	1,302
Avg. words per notification	9.3
Privacy tokens replaced	76

A. Task Definition and Ranking

We frame urgency detection as a 3-class text classification task. For each notification, the model predicts:

$$y \in \{\text{High}, \text{Medium}, \text{Low}\}.$$

High means urgent or time-sensitive (needs attention soon), Medium means important but can wait, and Low means not urgent (often informational or promotional). This setup follows prior work showing notifications vary in importance and can negatively affect attention when they are not managed well [1], [6], [12].

In addition to predicting a class label, we also create a ranking score so we can sort notifications by urgency. We compute:

$$\text{PriorityScore} = P(\text{High}) + 0.5 \cdot P(\text{Medium}),$$

and rank notifications from highest to lowest PriorityScore. This reflects the real use case: the system should surface the most urgent notifications first.

B. Dataset Collection (Week 5)

We curated our own dataset of real notifications collected by the group. The raw dataset file contained 742 notifications. Each record includes: `timestamp`, `app_name`, `notif_type`, `notif_content`, and `urgency`. After cleaning, the final dataset contains 737 notifications (5 removed due to empty content).

The curated dataset and preprocessing pipeline are publicly available [16].

The urgency label is one of three classes: High, Medium, or Low. The overall label distribution is: 258 High (35.0%), 196 Medium (26.6%), and 283 Low (38.4%). This distribution is shown in Fig. 1. We also summarize the dataset statistics in Table I.

As shown in Fig. 1, the three urgency classes are reasonably balanced, with slightly more Low urgency notifications. This supports using macro-F1 for evaluation and stratified splitting.

C. Cleaning and Privacy-Preserving Anonymization (Week 5)

Because our dataset contains real notification text, privacy is a priority. Before training any model, we applied a privacy-preserving anonymization pipeline to the notification text.

First, we removed rows where `notif_content` was empty. Then we normalized whitespace (extra spaces, line

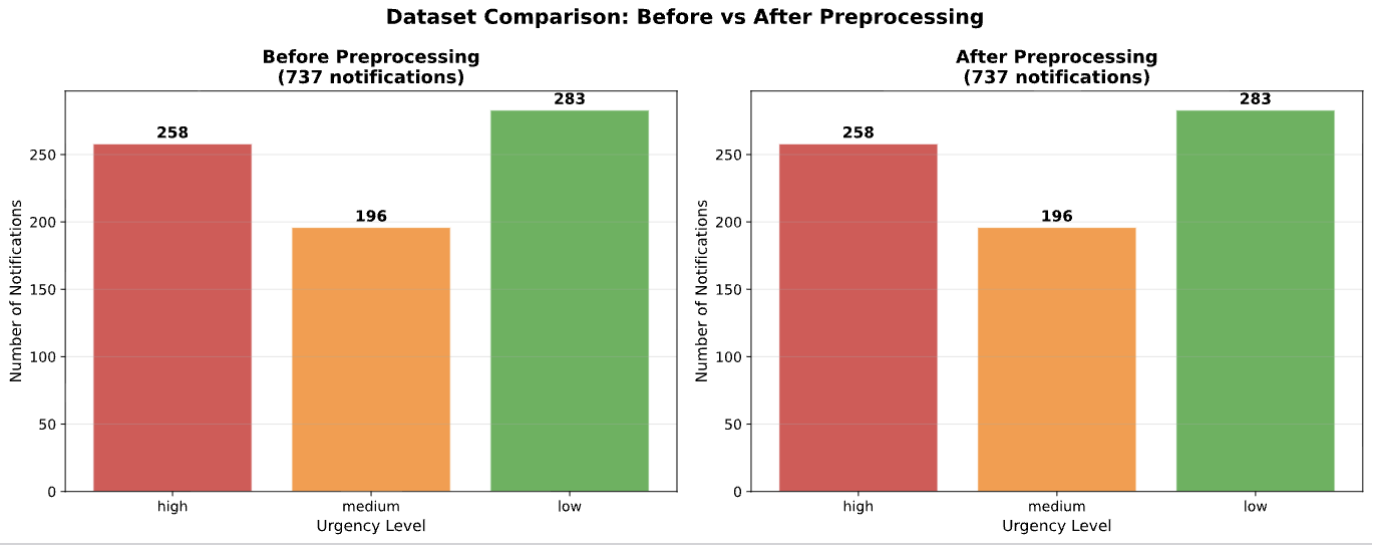

Fig. 1. Urgency distribution in our dataset (737 notifications).

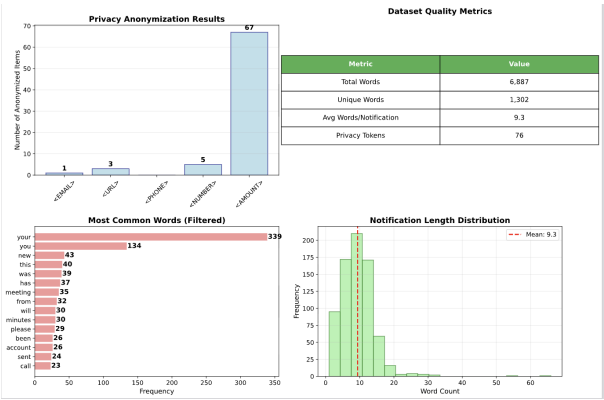

Fig. 2. Privacy anonymization results and dataset quality statistics.

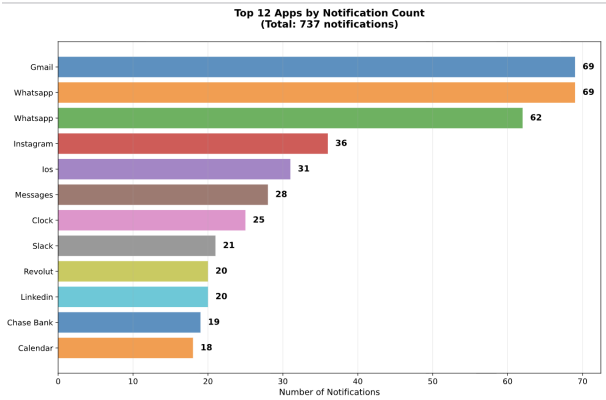

Fig. 3. Top apps by notification count in the curated dataset.

breaks). After that, we masked common sensitive patterns using placeholders: <EMAIL> for emails, <URL> for links, <PHONE> for phone numbers, <AMOUNT> for money amounts, and <NUMBER> for long numeric codes (4+ digits). This step reduces the chance of exposing personal information and also prevents the model from learning urgency from specific identifiers instead of meaning.

In total, 76 privacy tokens were masked across the dataset, with the most frequent being <AMOUNT> (67 occurrences). Fig. 2 shows the anonymization summary and basic dataset quality metrics.

D. Exploratory Data Analysis (Preprocessing Support)

We ran simple exploratory analysis to understand the dataset and make preprocessing choices. Most notifications are short (average 9.3 words per message). High and Medium urgency messages are slightly longer on average than Low urgency messages, which is expected because urgent messages often include more details (e.g., time, action, or failure reason). These length patterns helped us choose a reasonable maximum sequence length for tokenization.

We also checked which apps appear most often to understand possible dataset bias (for example, if a few apps dominate the dataset). Fig. 3 can be included as a descriptive plot of the most frequent apps.

E. Train/Validation/Test Split (Week 6)

To evaluate fairly, we created stratified splits so each urgency class keeps the same proportions across train/validation/test. Using a fixed random seed of 42, we split the dataset into: 519 training notifications (70.4%), 109 validation notifications (14.8%), and 109 test notifications (14.8%). Fig. 4 shows the split sizes and confirms the class proportions stay consistent.

F. Text Preprocessing (Week 6)

We keep preprocessing simple so we do not remove useful urgency cues. After anonymization, we keep punctuation and

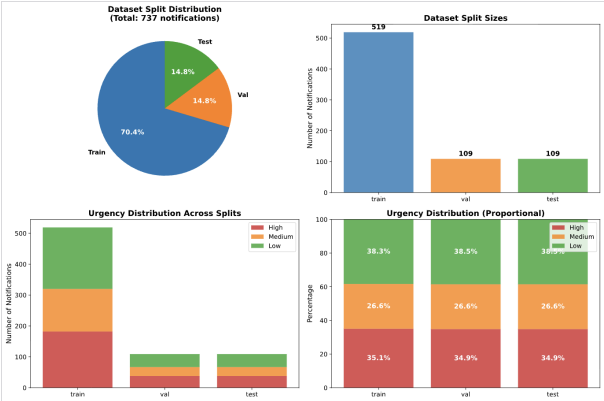

Fig. 4. Stratified train/validation/test split and urgency distribution across splits.

casing (e.g., "URGENT", exclamation marks) because they may carry urgency signals.

For transformer models, we tokenize using the pretrained model tokenizer [13]. Since notifications are short, we use a maximum sequence length between 64 and 128 tokens and truncate longer messages. This covers nearly all notifications while keeping training efficient.

For baseline models, we apply basic normalization (lowercasing and whitespace cleanup) and represent the text with word-based features (e.g., TF-IDF). This baseline is important to check whether transformer models actually improve performance beyond simple keywords.

G. Models

We compare a baseline model to transformer-based models.

Baseline: a simple bag-of-words/TF-IDF representation with a lightweight classifier. This is fast, easy to interpret, and gives a strong reference point.

Transformer models: our main approach fine-tunes pretrained transformer encoders for 3-class text classification. We fine-tune a BERT-style model [13]. If time allows, we also compare against RoBERTa, which often performs better on classification tasks due to improved pretraining [14]. Training and evaluation are implemented with the Transformers library [15].

We use transformers because urgency is often expressed through short phrases and context (for example: "verify now", "meeting in 10 minutes", "payment failed"). Transformers capture meaning better than keyword-only methods [13], [14].

H. Training Setup

We fine-tune models using the training split and select hyperparameters using the validation split. We start with standard settings for transformer fine-tuning: 3–5 epochs, batch size around 16, and learning rate around 2×10^{-5}. We use early stopping based on validation macro-F1 to reduce overfitting.

If class imbalance affects results, we use class-weighted loss or balanced sampling so the model does not learn to predict only the majority class.

I. Evaluation

We evaluate using accuracy, precision, recall, and F1-score, with macro-F1 as the main metric because it treats all urgency classes equally.

Since the system is meant to prioritize urgent notifications, we also evaluate ranking quality. Using PriorityScore, we sort notifications and compute a top-k check, such as Precision@10 (how many of the top 10 notifications are truly High). This directly tests whether urgent notifications are surfaced first.

J. Bias and Ethical Considerations

Notifications and urgency can vary across users, contexts, and apps [5], [6]. Because our dataset is collected by a small group, it may not represent all users equally. To reduce bias risks, we report the app distribution (Fig. 3) and check whether a few apps dominate the dataset. We also keep privacy protections through anonymization and do not include raw personal identifiers in the final paper.

Prior work shows frequent interruptions can harm attention and performance [1], [8], [12]. Our goal is to reduce unnecessary distraction, so we avoid creating a system that simply labels everything as urgent. Using macro-F1 and ranking evaluation helps confirm the model distinguishes urgency levels rather than over-predicting High.

IV. Results

V. Discussion and Future Work

Acknowledgment

None.

REFERENCES

[1] B. P. Bailey and J. A. Konstan, "On the need for attention-aware systems: Measuring effects of interruption on task performance, error rate, and affective state," *Computers in Human Behavior*, vol. 22, no. 4, pp. 685–708, 2006. Available: https://doi.org/10.1016/j.chb.2005.12.009

[2] S. T. Iqbal and E. Horvitz, "Notifications and awareness: A field study of alert usage and preferences," in *Proc. CSCW*, 2010, pp. 27–30. Available: https://doi.org/10.1145/1718918.1718926

[3] M. Pielot, K. Church, and R. de Oliveira, "An in-situ study of mobile phone notifications," in *Proc. MobileHCI*, 2014, pp. 233–242. Available: https://doi.org/10.1145/2628363.2628364

[4] A. Shirazi, N. Henze, T. Dingler, M. Pielot, D. Weber, and A. Schmidt, "Large-scale assessment of mobile notifications," in *Proc. CHI*, 2014, pp. 3055–3064. Available: https://doi.org/10.1145/2556288.2557189

[5] A. Mehrotra, V. Pejovic, J. Vermeulen, R. Hendley, and M. Musolesi, "My phone and me: Understanding people's receptivity to mobile notifications," in *Proc. CHI*, 2016, pp. 1021–1032. Available: https://doi.org/10.1145/2858036.2858566

[6] A. Visuri, K. van Berkel, C. Luo, J. Goncalves, S. Hosio, and D. Ferreira, "Understanding smartphone notifications' user interactions and content importance," *International Journal of Human-Computer Studies*, vol. 128, pp. 49–61, 2019. Available: https://doi.org/10.1016/j.ijhcs.2019.03.001

[7] B. Esteves, K. Fraser, S. Kulkarni, O. Conlan, and V. Rodríguez-Doncel, "Now, later, never: A study of urgency in mobile push-notifications," in *Advances in Mobile Computing and Multimedia Intelligence*, Springer, 2022. Available: https://doi.org/10.1007/978-3-031-20436-4_4

[8] J. Ho and S. S. Intille, "Using context-aware computing to reduce the perceived burden of interruptions from mobile devices," in *Proc. CHI*, 2005, pp. 909–918. Available: https://doi.org/10.1145/1054972.1055100

[9] A. Mehrotra, R. Hendley, and M. Musolesi, "PrefMiner: Mining user's preferences for intelligent mobile notification management," in *Proc. UbiComp*, 2016, pp. 1223–1234. Available: https://doi.org/10.1145/2971648.2971747

[10] M. Pielot, R. de Oliveira, H. Kwak, and N. Oliver, "Didn't you see my message? Predicting attentiveness to mobile instant messages," in *Proc. CHI*, 2014, pp. 3319–3328. Available: https://doi.org/10.1145/2556288.2556973

[11] M. Pielot, K. Church, and R. de Oliveira, "Beyond interruptibility: Predicting opportune moments to engage mobile phone users," *Proc. ACM IMWUT*, vol. 1, no. 3, 2017. Available: https://doi.org/10.1145/3130956

[12] K. Kushlev, J. D. E. Proulx, and E. W. Dunn, "Silence your phones: Smartphone notifications increase inattention and hyperactivity symptoms," in *Proc. CHI*, 2016, pp. 1011–1020. Available: https://doi.org/10.1145/2858036.2858359

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186. Available: https://aclanthology.org/N19-1423/

[14] Y. Liu, M. Ott, N. Goyal, et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019. Available: https://arxiv.org/abs/1907.11692

[15] T. Wolf, L. Debut, V. Sanh, et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP (System Demonstrations)*, 2020, pp. 38–45. Available: https://aclanthology.org/2020.emnlp-demos.6/

[16] O. J. Holmes, D. S. Ivanov, S. C. B. San José, and G. N. F. de Córdoba Javronich, "Curated Smartphone Notification Dataset for Urgency Classification," IE University, 2026. Available: https://github.com/sofiasanjose/Notification-Priority-Classification-Using-Semantic-Analysis